UNIVERSIDAD AUTÓNOMA DE CIUDAD JUÁREZ

Instituto de Ingeniería y Tecnología

Departamento de Ingeniería Eléctrica y Computación



Integración y entrega continua para sistemas legado en producción: contenedorización y entornos de prueba bajo demanda sobre infraestructura autoalojada

Alumno:

Oscar Alonso Nava Rivera 214729

Requisito para la obtención del título de

INGENIERO EN SOFTWARE

Director:
Mtro. Abraham López Najera

Ciudad Juárez, Chihuahua 21 de noviembre de 2026

Introducción

[Una cuartilla: 1) campo de investigación, 2) lo ya estudiado por otros, 3) área de oportunidad, 4) objetivo del proyecto, 5) justificación.]

1. Planteamiento del problema

Una organización del sector público mantiene su sistema principal, el cual sostiene gran parte de la operación administrativa y operativa e integra las diferentes áreas de esta organización. La información se almacena en dos motores de base de datos, uno relacional y otro documental. El segundo, al no ser relacional, no implementa llaves foráneas como restricción del propio motor y, al tratarse de motores distintos, tampoco hay un mecanismo de integridad referencial que los conecte entre sí, por lo que la relación entre los datos depende de una convención y no de una restricción verificada por el sistema.

El sistema nació hace tres años orientado principalmente a elementos operativos, con funciones como la generación de resguardos de unidad y bitácoras de servicio. Con el tiempo escaló hacia el área administrativa de la organización, incorporando de forma progresiva nuevas funcionalidades hasta convertirse en el sistema principal de la organización, sin que su arquitectura se revisara al mismo ritmo que su alcance. Actualmente sostiene operaciones críticas de la organización, mientras que su proceso de despliegue continúa siendo completamente manual.

Para realizar un despliegue es necesario acceder mediante SSH al servidor donde se aloja la aplicación, actualizar el código mediante operaciones de Git sobre el repositorio, ejecutar manualmente las migraciones pendientes y limpiar cachés de configuración y rutas, sin mecanismos de verificación automática. El historial del repositorio refleja un volumen considerable de cambios: desde su inicio se han fusionado 563 solicitudes de cambio en la rama principal, sobre aproximadamente 8,000 commits. Estas integraciones pueden incluir cambios que deben aplicarse sobre los motores de base de datos: modificación de esquemas en el motor relacional y, en el motor no relacional, que no tiene esquema fijo, operaciones equivalentes como la creación de índices; en ambos casos deben identificarse y ejecutarse manualmente y en el orden correspondiente.

El ritmo de integración de cambios a la rama principal no equivale a la frecuencia real de despliegue: con regularidad se agrupan varios cambios fusionados antes de llevarlos al servidor, aunque no siempre ocurre así ni existe un criterio fijo de cuántos se acumulan. Tampoco existe un registro sistemático del tiempo entre la fusión de un cambio a la rama principal y su despliegue real en producción; de forma informal se sabe que ese tiempo es corto, ya que no se deja código fusionado sin desplegar por periodos prolongados, pero no hay un dato preciso que lo respalde. Tampoco se conoce, con el mismo nivel de precisión, qué proporción de despliegues requiere una corrección posterior.

El proceso manual presenta dos modos de falla asociados a la ejecución de pasos a mano. El primero, ya observado de forma recurrente, es olvidar limpiar las cachés de configuración y rutas al final de cada despliegue, lo que deja el sistema sirviendo una versión desactualizada hasta que se detecta el problema. El segundo, un riesgo identificado pero sin evidencia de haberse materializado, es específico de los despliegues que agrupan varios cambios: omitir alguna migración o seeder de los distintos cambios acumulados.

1.1. Antecedentes

El primer elemento a considerar al intervenir un proceso de entrega de software es establecer un mecanismo sistemático y validado para medir su desempeño antes y después de la automatización. Forsgren, Humble y Kim [1] sintetizaron años de investigación empírica en miles de organizaciones y aislaron cuatro métricas clave, conocidas en la literatura como métricas DORA: frecuencia de despliegue (*deployment frequency*), tiempo de entrega de los cambios (*lead time for changes*), tiempo de restauración del servicio (*time to restore service*) y tasa de fallos por cambio (*change failure rate*). El hallazgo central demuestra que las organizaciones de alto desempeño no sacrifican estabilidad por velocidad, sino que consiguen ambas de forma simultánea. Para este trabajo, estas métricas proporcionan un vocabulario y una definición operacional ya validados, ofreciendo un marco científico para cuantificar la mejora respecto a la línea base (*baseline*) de los despliegues manuales.

Sobre la adopción de Integración y Entrega Continua (CI/CD), existe un consenso sólido respecto a las principales barreras de implementación. Revisiones sistemáticas de literatura empírica desarrolladas por Lukkanen et al. [2] y Proulx et al. [3] coinciden en que los factores humanos y organizacionales superan holgadamente a los problemas técnicos en la generación de fallas durante la liberación de código. Estos hallazgos respaldan directamente el diagnóstico del sistema de estudio de este proyecto: la omisión de limpieza de cachés tras un despliegue manual constituye una falla observada de forma recurrente, mientras que la omisión de migraciones al agrupar múltiples *commits* o cambios constituye un riesgo potencial identificado, sin evidencia de haberse materializado. Ambos problemas son consistentes con la influencia de factores humanos. La literatura señala que la automatización mediante un *pipeline* establece un mecanismo restrictivo de construcción que puede reducir la posibilidad de errores humanos por omisión al hacer explícitos y verificables los pasos del despliegue.

Al intentar aplicar estos principios a sistemas *legacy*, la literatura técnica se inclina hacia una refactorización profunda o la descomposición en microservicios. Sin embargo, diferentes estudios recientes en la ingeniería de software empresarial advierten sobre los elevados riesgos y costos que puede ocasionar esta estrategia, especialmente en infraestructura restringida y autoalojada. La práctica documentada respalda la modernización incremental [4], un enfoque que prioriza la contenedorización de la aplicación tal como está, aislando sus dependencias de tiempo de ejecución (como el *runtime* de la aplicación) sin reescribir la lógica del negocio. Este es un paso crítico en contextos institucionales que operan sobre infraestructura propia y carecen de orquestación en la nube pública.

Un desafío arquitectónico particular del sistema en cuestión es su dependencia de un esquema de base de datos dual y heterogéneo (un motor relacional y uno documental simultáneos), desprovisto de un mecanismo nativo de integridad referencial. Estudios empíricos sobre entrega continua, como el de Shahin et al. [5], identifican la sincronización y la evolución de esquemas de bases de datos como uno de los cuellos de botella técnicos más severos al automatizar despliegues. Dado que no existe una gestión de transacciones distribuidas entre ambos motores, cualquier estrategia de automatización debe tratar ambas bases como una unidad lógica unificada al ejecutar migraciones o *backfills* de datos. Esto impone una restricción de diseño severa para el *pipeline* propuesto.

Por otra parte, el obstáculo más significativo frente a la literatura tradicional de CI/CD es la validación del código. La inmensa mayoría de los trabajos académicos asumen la preexistencia de una suite de pruebas automatizadas como el *quality gate* (compuerta de calidad) principal. Cuando

un sistema *legacy* carece de pruebas unitarias o de integración, forzar la entrega continua presenta un riesgo crítico. Para cerrar esta brecha en escenarios con datos altamente acoplados, la industria sugiere la instanciación de entornos de prueba bajo demanda (*on-demand preview environments*), nutridos con *dumps* sanitizados de producción. Esto transfiere la responsabilidad de la validación del *pipeline* a un entorno aislado donde se puede realizar una comprobación funcional manual de alta fidelidad.

El contexto operativo institucional del sector público con servidores propios (*on-premise*) añade una capa final de complejidad debido a restricciones presupuestales y la falta de registros de contenedores (*container registries*) preexistentes. Un precedente altamente relevante en este rubro es el trabajo de Morales Agredo [6] en la Pontificia Universidad Javeriana Cali, quien diseñó una propuesta de CI/CD orientada a aplicaciones *legacy* universitarias. Su investigación demuestra que la transición hacia flujos automatizados con plataformas autoalojadas (*self-hosted*) es viable e imperativa en organizaciones con infraestructura limitada, sin depender de proveedores externos.

Cruzando los ejes anteriores emerge con claridad la brecha técnica y operativa que motiva este trabajo. La literatura revisada documenta sólidamente cómo evaluar el desempeño, los riesgos del error humano, la viabilidad de la contenedorización incremental y la necesidad de modernización en instituciones públicas. Ninguno de estos trabajos, sin embargo, aborda de manera conjunta la automatización de la entrega de un sistema monolítico con un esquema de datos heterogéneo, operado sobre infraestructura *on-premise* sin acceso a nube pública, y que deba garantizar la seguridad del despliegue en ausencia total de una suite de pruebas automatizadas. Es precisamente en la resolución simultánea de estas restricciones operativas donde se inscribe la contribución de este proyecto.

1.2. Definición del problema

El problema central que aborda este trabajo radica en la inestabilidad y el riesgo técnico generados por la gestión manual del ciclo de liberación de software en un entorno monolítico en producción. La ejecución directa de rutinas de despliegue y configuración sobre la infraestructura en vivo, sumada a la imposibilidad de validar los cambios en entornos aislados de forma previa, expone la operación a fallas recurrentes inducidas por el factor humano y compromete la confiabilidad técnica de cada integración.

1.3. Justificación

La automatización del ciclo de entrega de software no es únicamente una medida para incrementar la velocidad en el desarrollo, sino un mecanismo que permite mitigar riesgos operativos. En el contexto de este proyecto, la ejecución de despliegues manuales sobre una infraestructura *on-premise* expone a la organización a posibles interrupciones del servicio derivadas de errores en la configuración, como la falta de sincronización al aplicar migraciones o la simple omisión en la limpieza de cachés. Al carecer de una suite de pruebas automáticas, el riesgo de integrar cambios defectuosos a producción es elevado.

Resolver este problema genera un impacto operativo directo: al sustituir las intervenciones manuales por un *pipeline* declarativo y generar entornos de prueba bajo demanda, se crea una “com-

puerta de calidad" (*quality gate*) que aísla los fallos antes de que alcancen a los usuarios finales. Esto beneficia directamente al equipo técnico, al reducir la dependencia de procedimientos manuales susceptibles a errores de omisión o ejecución, y al favorecer la estandarización, reproducibilidad y trazabilidad del proceso de liberación. Asimismo, beneficia a la institución, al disminuir el riesgo de interrupciones durante los despliegues y contribuir a la continuidad operativa de un sistema que sustenta su gestión administrativa.

1.4. Objetivo general

Diseñar y validar empíricamente una arquitectura de integración y entrega continua (CI/CD) adaptada a sistemas *legacy*, mediante contenedorización y un entorno de prueba bajo demanda, con el propósito de mitigar la vulnerabilidad operativa y reducir las intervenciones manuales rutinarias en la infraestructura de producción de un monolito operado sobre un esquema de base de datos dual y heterogéneo.

1.5. Objetivos específicos

Para alcanzar el objetivo general propuesto, se plantea estructurar las siguientes metas parciales:

- Aislar el entorno de ejecución de la aplicación mediante la adopción de imágenes de contenedor inmutables, preservando la lógica del negocio del monolito sin recurrir a una reescritura de código orientada a microservicios.
- Orquestar un mecanismo de aprovisionamiento declarativo de entornos de prueba efímeros bajo demanda, alimentados por volcados de datos sanitizados del esquema heterogéneo (MySQL y MongoDB) de producción, para proveer un entorno de validación de alta fidelidad.
- Diseñar e implementar un *pipeline* de Integración Continua (CI) sobre infraestructura autoalojada, que incorpore compuertas de calidad (*quality gates*) basadas en análisis estático avanzado, para verificar automáticamente cada cambio integrado al repositorio y detectar defectos, vulnerabilidades y *code smells* antes de su despliegue, ante la ausencia histórica de una suite de pruebas automatizadas.
- Evaluar comparativamente la resiliencia operativa de la arquitectura propuesta frente a la línea base de despliegues manuales, cuantificando la mejora a través de métricas estandarizadas (*deployment frequency* y *change failure rate*).

1.6. Hipótesis

[Opcional según el tipo de trabajo.]

1.7. Alcances y limitaciones

[Qué sí y qué no cubre el proyecto.]

2. Marco teórico

[Teoría, conceptos y trabajo relacionado que fundamentan el proyecto. Una subsección por concepto, cada una introducida con un párrafo.]

2.1. Subtema

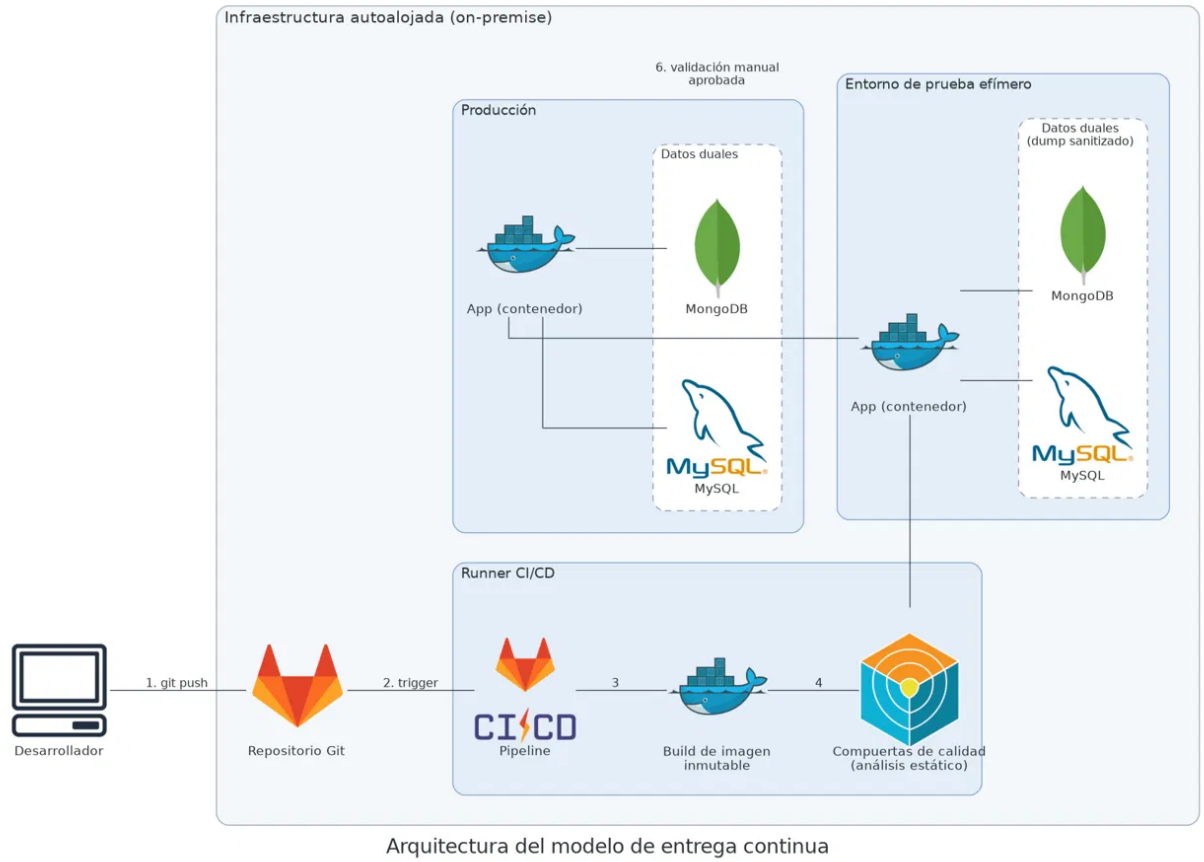
2.1.1. Subsubtema

3. Producto propuesto

El entregable principal de este proyecto no se limita a la simple configuración de una herramienta de software, sino que constituye un modelo arquitectónico de referencia para la modernización de la entrega de valor en sistemas legados. La solución transformará el actual paradigma de despliegue artesanal, fragmentado y vulnerable en una cadena de suministro de software predecible, auditable y basada en principios de Infraestructura como Código (IaC).

Las innovaciones tecnológicas y operativas que caracterizan a esta propuesta incluyen:

- **Estándar de inmutabilidad:** Transición hacia contenedores estandarizados que erradican la divergencia de configuración entre los entornos de desarrollo local y los servidores en vivo.
- **Orquestación declarativa en entornos restringidos:** Implementación de un flujo de trabajo CI/CD integral operado enteramente en *on-premise*, demostrando la viabilidad de la cultura DevOps sin depender de proveedores de nube pública.
- **Entornos de validación de alta fidelidad:** Resolución del déficit de pruebas automatizadas mediante la instanciación de replicas aisladas del sistema. Estas instancias, poblados con datos representativos del entorno de produccion, habilitaran la validación funcional manual en condiciones similares a producción antes de la fusión del código en la rama principal.
- **Transaccionalidad en esquemas heterogéneos:** Automatización rigurosa de las migraciones de base de datos que sincroniza los cambios estructurales entre motores (MySQL y MongoDB) como una única unidad lógica de despliegue.



Arquitectura del modelo de entrega continua

Figura 1: Arquitectura propuesta para la entrega continua.

4. Metodología

[Pasos o fases para alcanzar el objetivo, cada uno con definición y descripción de cómo se realizará.]

5. Programa de actividades

[Cronograma de actividades por periodo.]

6. Resultados esperados

[Qué se espera obtener al terminar.]

Referencias

[1] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. Portland, OR: IT Revolution Press, 2018.

[2] E. Laukkanen, J. Itkonen, and C. Lassenius, "Problems, causes and solutions when adopting continuous delivery—a systematic literature review," *Information and Software Technology*, vol. 82, pp. 55–79, 2017.

[3] A. Proulx, F. Raymond, B. Roy, and F. Petrillo, "Problems and solutions of continuous deployment: A systematic review," 2018. Preprint, no arbitrado.

[4] J. Kosi'nska and K. Zielinski, "Cloud-native enterprise engineering: design, automation, and continuous delivery," *Software: Practice and Experience*, vol. 50, no. 8, pp. 1415–1443, 2020.

[5] M. Shahin, M. A. Babar, and L. Zhu, "Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices," *IEEE Access*, vol. 5, pp. 3909–3943, 2017.

[6] A. Morales Agredo, "Propuesta de implementaci'on de integraci'on continua (ci) y despliegue continuo (cd) en aplicaciones *legacy* de la pontificia universidad javeriana cali," trabajo de grado, Pontificia Universidad Javeriana, Cali, Colombia, 2021.

A. Apéndice

[Material secundario o demasiado extenso para el cuerpo: código fuente completo, derivaciones, tablas de resultados completos, convenios, etc. Va después de la literatura citada y no lleva introducción ni conclusión.]